

Multi-Server Fan-out & Health, Worked Out by Hand

A MultiClient that queries N servers can wait for them serially (sum of latencies) or concurrently (max). We derive the speed-up and the health metrics that decide when to drop a flaky server.

What this document does. A client that aggregates N servers must list tools, route calls, and stay healthy when one server flakes. This document derives the latency of fanning a request to N servers serially versus concurrently (sum vs max), computes the speed-up, and works out the health signals — error rate and an exponentially-weighted latency average — that drive graceful degradation.

Concepts covered: the pending-future map · serial vs parallel fan-out · sum-vs-max latency · parallel speed-up · error rate · exponential moving average (EMA).

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: list tools across four servers

On startup the client calls `tools/list` on $N = 4$ servers with these round-trip latencies:

$$W = (W_1, W_2, W_3, W_4) = (40, 90, 35, 220) \text{ ms.}$$

Server 4 is slow (a remote HTTP backend); the rest are local. The client holds a pending map $\{\text{id} \mapsto \text{Future}\}$ so it can have all four requests outstanding at once.

Intuition

Because the `id` field lets responses return in any order (Module 1.0), the client doesn't have to wait for server 1 before asking server 2. It can fire all N requests, then **await** the whole set. Whether it actually does that — or loops one at a time — is the difference between max and $\sum$.

2 Step 1 — Serial fan-out: latencies add

A naive client awaits each server before starting the next. Total time is the sum:

$$W_{\text{serial}} = \sum_{i=1}^N W_i = 40 + 90 + 35 + 220 = 385 \text{ ms.}$$

Every server, even the fast ones, waits behind the one ahead of it.

3 Step 2 — Parallel fan-out: latency is the slowest

Fire all N requests, then await them together. The batch finishes when the *last* one returns:

$$W_{\text{parallel}} = \max_i W_i = \max(40, 90, 35, 220) = 220 \text{ ms.}$$

The three fast servers overlap entirely under the slow one’s 220 ms — their time is hidden, not added.

$$\text{speed-up} = \frac{W_{\text{serial}}}{W_{\text{parallel}}} = \frac{385}{220} = 1.75 \times .$$

strategy	latency	formula
serial	385 ms	$\sum W_i$
parallel	220 ms	$\max W_i$
speed-up	$1.75 \times$	$\sum / \max$

Intuition

Parallel fan-out is bounded below by the *slowest* server — you can never beat $\max W_i$ no matter how many fast servers you add. The speed-up $\sum / \max$ is largest when latencies are similar (many servers each contributing) and smallest when one server dominates (here the 220 ms server caps the win at $1.75 \times$). One slow server sets the pace for everyone.

Watch out

Parallel fan-out also means one hung server can stall the whole batch up to its timeout. Always bound each request with a per-call timeout τ , so $W_{\text{parallel}} = \min(\max_i W_i, \tau)$ — otherwise a dead server turns max into “forever.”

4 Step 3 — Health: error rate

To degrade gracefully the client tracks each server’s error rate over the last n calls:

$$\text{error rate} = \frac{\text{failures}}{\text{total}}.$$

Server 4 over its last 20 calls: 5 failures $\Rightarrow 5/20 = 0.25$. If the policy is “drop a server above 20% error,” server 4 is now ejected from the aggregated tool list and its tools disappear from the catalogue until it recovers.

5 Step 4 — Health: exponential moving average of latency

A single slow call shouldn’t condemn a server; a sustained climb should. An exponential moving average (EMA) weights recent samples more, with smoothing factor $\alpha \in (0, 1]$:

$$\text{EMA}_t = \alpha x_t + (1 - \alpha) \text{EMA}_{t-1}.$$

Take $\alpha = 0.3$, starting $\text{EMA}_0 = 220$ ms, with new samples 300, 280, 260:

$$\text{EMA}_1 = 0.3(300) + 0.7(220) = 244, \quad \text{EMA}_2 = 0.3(280) + 0.7(244) = 254.8, \quad \text{EMA}_3 = 0.3(260) + 0.7(254.8) = 255.8$$

The average climbs toward the new regime (~ 280) smoothly instead of jumping, so a single spike barely moves it while a sustained rise does.

Mini-example — this operation in isolation

α sets the memory. $\alpha = 1$ is “last sample only” (no smoothing, jumpy); $\alpha \rightarrow 0$ is “never forget the past” (sluggish). The effective window is $\approx 1/\alpha$ samples — $\alpha = 0.3$ remembers

roughly the last 3–4 calls. Pick α to match how fast you want to react to a degrading server without overreacting to one bad packet.

6 The argument, at a glance

#	Idea	Formula	Value
1	Serial fan-out	$\sum W_i$	385 ms
2	Parallel fan-out	$\max W_i$	220 ms
2	Speed-up	$\sum / \max$	1.75×
3	Error rate	failures/total	0.25
4	Latency EMA	$\alpha x_t + (1 - \alpha)\text{EMA}_{t-1}$	256.4 ms

7 Equation cheat-sheet

Serial fan-out latency	$\sum_i W_i$
Parallel fan-out latency	$\max_i W_i$ (with timeout: $\min(\max_i W_i, \tau)$)
Parallel speed-up	$\sum_i W_i / \max_i W_i$
Error rate	failures / total
Latency EMA	$\text{EMA}_t = \alpha x_t + (1 - \alpha)\text{EMA}_{t-1}$
EMA effective window	$\approx 1/\alpha$ samples

A Reference implementation

```

W = [40, 90, 35, 220]                # per-server latency (ms)
print(sum(W), max(W), round(sum(W)/max(W), 2)) # 385 220 1.75

def ema(samples, alpha, start):
    e = start
    for x in samples:
        e = alpha * x + (1 - alpha) * e
    return e

print(round(ema([300, 280, 260], 0.3, 220), 1)) # 256.4
print(5 / 20)                                # 0.25 error rate

```

— end of document —